The Journal of Open Source Software

Eunoia: Area-Proportional Euler and Venn Diagrams in Rust, Julia, Python, R, and JavaScript

Johan Larsson [1] **and Nicole M. Lee** [2]

1 Department of Mathematical Sciences, University of Copenhagen, Denmark **2** Program in Neuroscience and Behavioral Disorders, Duke-NUS Medical School, Singapore

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Eunoia is a Rust library (Larsson, 2026) for area-proportional Euler and Venn diagrams. Given the sizes of a collection of sets and their intersections, it fits a diagram of circles, ellipses, squares, or rectangles whose overlapping areas match those quantities as closely as possible. When an exact representation does not exist, Eunoia returns the fitted layout with residuals and goodness-of-fit statistics that help users assess how well it represents the data. The Rust crate also powers an R package (eulerr), a Python package, a Julia package, a JavaScript package compiled to WebAssembly, a C API for further bindings, and a web app built on the JavaScript package. All bindings call the same implementation and produce the same layout given the same specification, options, and seed.

Statement of Need

Euler diagrams (Euler, 1802) depict relationships between sets, and area-proportional Euler diagrams do so quantitatively: the area of each region is proportional to the size of the corresponding set intersection. They are common in the life sciences, for instance, where researchers use them to compare gene and protein lists across conditions or studies. For most specifications involving three or more sets, however, no exact area-proportional diagram exists (Wilkinson, 2012), and the diagram must instead be constructed numerically (Chow, 2007). Fitting a diagram is therefore an optimization problem: choosing the shapes' positions and parameters to minimize the discrepancy between fitted and requested intersection areas.

Existing alternatives offer different compromises. venneuler introduced multidimensional scaling (MDS) initialization followed by overlap-aware refinement (Wilkinson, 2012). venn.js improved that initialization and added exact area calculations (Frederickson, 2015). eulerAPE introduced ellipses for three-set diagrams (Micallef & Rodgers, 2014), and eulerr generalized them to any number of sets (Larsson & Gustafsson, 2018).

Eunoia extends eulerr by

- replacing its fitting algorithm with a multistage method that combines local and global optimization,
- supporting squares and rectangles, and
- providing analytical gradients.

Eunoia serves researchers and others who need trustworthy proportional set visualizations in R, Python, Julia, JavaScript, Rust, or C. Developers can also embed its diagram fitter and visualizer in their own tools, including interactive web visualizations.

DRAFT

The Journal of Open Source Software

State of the Field

Table 1 compares Eunoia with representative area-proportional diagram fitters: venneuler (Wilkinson, 2012), eulerAPE (Micallef & Rodgers, 2014), venn.js (Frederickson, 2015), matplotlib-venn (Tretyakov, 2024), nVenn (Pérez-Silva et al., 2018), Edeap (Wybrow et al., 2021), and eulerr (Larsson & Gustafsson, 2018).

Table 1: Related software for area-proportional Euler and Venn diagrams: fitting algorithm, supported shapes, maximum number of sets, and implementation language.

Package	Algorithm	Shapes	Sets	Language
`venneuler`	Stress minimization	Circles	Any	`Java`, `R`
`eulerAPE`	Hill climbing	Ellipses	3	`Java`
`venn.js`	MDS + final layout	Circles	Any	`JavaScript`
`matplotlib-venn`	Analytic placement	Circles	3	`Python`
`nVenn`	Physics-based simulation	Polygons	Any	`C++`, `R`
`Edeap`	Hill climbing	Ellipses	Any	`JavaScript`
`Eunoia`	MDS + final layout	Circles, ellipses, squares, rectangles	Any	`Rust`, `R`, `Julia`, `Python`, `C`, `JS`

Eunoia is a separate library because eulerr's C++ implementation was coupled to R and could not directly support WebAssembly and the other bindings. Centralizing fitting, diagnostics, and geometry in an independent Rust crate keeps the interfaces consistent and avoids duplicating this work across bindings.

Example

A diagram specification names the sets and their intersections and gives the size of each region. For example, in Rust:

```rust
use eunoia::geometry::shapes::Ellipse;
use eunoia::{DiagramSpecBuilder, Fitter, InputType};

let spec = DiagramSpecBuilder::new()
    .set("SE", 13.0)
    .set("Treat", 28.0)
    .set("Anti-CCP", 101.0)
    .set("DAS28", 91.0)
    .intersection(&["SE", "Treat"], 1.0)
    .intersection(&["SE", "DAS28"], 14.0)
    .intersection(&["Treat", "Anti-CCP"], 6.0)
    .intersection(&["SE", "Anti-CCP", "DAS28"], 1.0)
    .input_type(InputType::Exclusive)
    .build()
    .unwrap();

let layout = Fitter::<Ellipse>::new(&spec).seed(1).fit().unwrap();
```

The four sets—SE, Treat, Anti-CCP, and DAS28—correspond to clinical variables in the rheumatoid arthritis data from Junta et al. (2009). The fitted layout is shown in Figure 1. Its diagError of 10^{-4} indicates that the diagram reproduces the data essentially exactly. The figure also shows Eunoia's automatic label placement: set labels and region quantities are

DRAFT

positioned at poles of inaccessibility of the region polygons, with labels placed outside the diagram when they do not fit inside their regions.

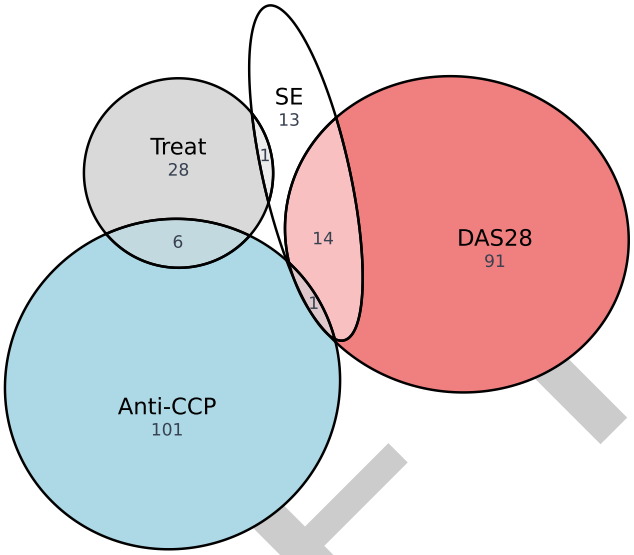


Figure 1: A four-set Euler diagram fitted with ellipses, with region quantities shown. Eunoia fits it to a diagError of 10^{-4}, an essentially exact diagram.

The bindings mirror this interface; the equivalent call in Python is:

```python
import eunoia as eu

diagram = eu.euler(
    {
        "SE": 13,
        "Treat": 28,
        "Anti-CCP": 101,
        "DAS28": 91,
        "SE&Treat": 1,
        "SE&DAS28": 14,
        "Treat&Anti-CCP": 6,
        "SE&Anti-CCP&DAS28": 1,
    },
    shape="ellipse",
)
```

The JavaScript, Julia, and R packages follow the same pattern.

Eunoia also constructs Venn diagrams, in which every intersection is drawn regardless of whether it is empty (Figure 2).

DRAFT

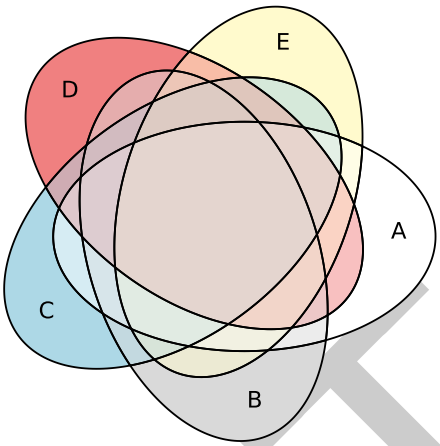


Figure 2: A canonical five-set Venn diagram drawn with ellipses.

Software Design

The library uses a trait-based design, with shapes selected at compile time. A given specification can be fitted with circles, ellipses, squares, or rectangles, as illustrated in Figure 3.

DRAFT

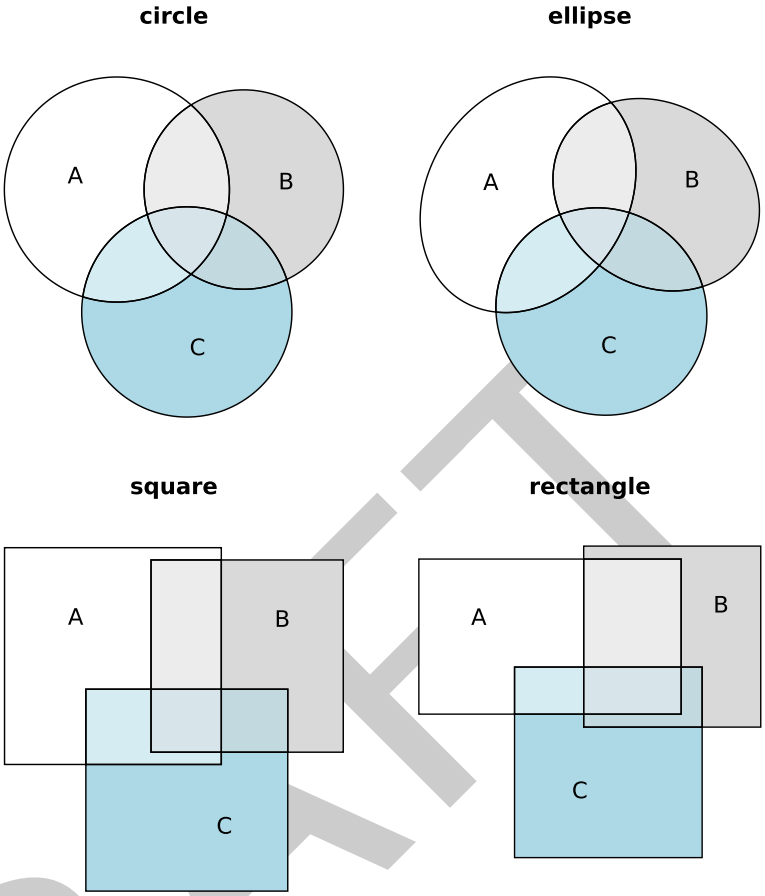


Figure 3: A three-set specification fitted with circles, ellipses, squares, and rectangles. Not every shape family can represent the input exactly: the diagError is 1.2×10^{-2} for circles, 1.7×10^{-2} for squares, and 4.5×10^{-2} for rectangles, compared to 2.3×10^{-14} for ellipses.

DRAFT

Fitting proceeds in two phases, roughly following the approach of venneuler (Wilkinson, 2012), venn.js (Frederickson, 2015), and the original eulerr package (Larsson & Gustafsson, 2018). The first phase uses multidimensional scaling to place shapes of fixed size so that their pairwise distances approximate those required by the pairwise intersections. For disjoint and contained set pairs, it uses the relaxed loss suggested by Frederickson (2015). The second phase refines all shape parameters to minimize a loss computed from the differences between the fitted and requested region areas[1]. Eunoia computes region areas analytically for every shape. It finds ellipse intersections through a projective-conic construction (Richter-Gebert, 2011) and computes the resulting overlaps from circular and elliptical segments in closed form (Eberly, 2016). For several shape families, the smooth losses have exact analytical gradients.

For shapes with analytical gradients, refinement begins with Levenberg-Marquardt least squares, with a bounded Levenberg-Marquardt retry if the loss remains above a threshold. If both local fits remain above the threshold, a bounded variant of CMA-ES (Hansen & Ostermeier, 1996) searches for another basin, which bounded Levenberg-Marquardt then polishes. Eunoia keeps the solution with the lowest loss. The pipeline uses several seeded restarts, which can run in parallel when the optional `parallel` feature is enabled (but remain serial in WebAssembly). For small set counts, one restart begins from a canonical Venn layout. Non-smooth losses—such as the maximum absolute region error—use derivative-free methods (Nelder–Mead or mesh-

[1]The default loss is a normalized sum of squared errors.

adaptive direct search) or can be replaced by a smooth surrogate.

Eunoia returns the fitted layout with its residuals, loss, and two goodness-of-fit statistics: stress, a venneuler-style normalized least-squares measure, and diagError, the maximum absolute difference between requested and fitted region shares. These diagnostics allow users to assess the diagram's quality numerically. A plotting module extracts the region polygons through polygon clipping and computes label positions. The R, Python, Julia, and JavaScript packages use this output for rendering. The core has no dependency on any host language's runtime and compiles to WebAssembly. The web app therefore fits diagrams entirely in the browser without a server.

Annotation and Interactivity

Area-proportional Euler diagrams encode the size of each set intersection as the area of a region. Eunoia adds three complementary features: unit glyphs show how many elements a region contains, member labels identify those elements, and interactive output reveals information about regions too small to annotate (Figure 4).

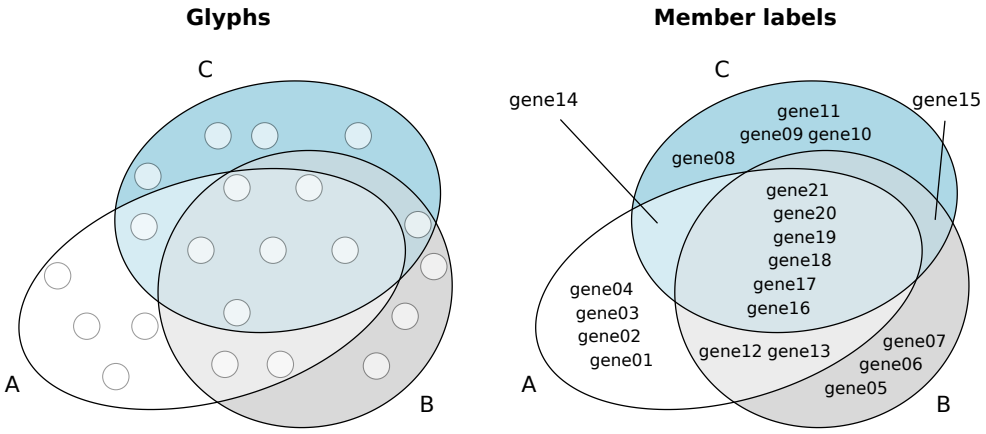


Figure 4: A three-set Euler diagram fitted with ellipses, shown with glyphs (left) and member labels (right). Callouts identify members in regions too narrow for their names.

Glyphs

Glyphs are circular marks, one per data unit, packed inside their regions following the eulerGlyphs approach of Micallef et al. (2012). They can also be added to Venn diagrams. All glyphs in a diagram are the same size, so counts can be compared across regions. By default, Eunoia searches for the largest size that fits all the glyphs in every region. The placement routines also accept a fixed size and report any glyphs that do not fit. Glyphs can be kept clear of the diagram's labels where space allows.

The shared placement routines offer four arrangements: a hexagonal grid (the default), a rectangular grid, an even, deterministic scatter, and a random scatter.

Member Labels

Member labels show the names of the elements in each region. The shared packing routine places each name individually inside its region and reduces all font sizes by the same factor

when space is limited, down to a minimum size. Names that still do not fit are reported as unplaced.

Interactivity

The Python package provides an interactive Plotly backend through `fit.plot_plotly()`. Each region has a hover tooltip showing its intersection label, sample size, and members. These tooltips are primarily useful for dense multi-set diagrams, in which small regions cannot hold any text. The JavaScript SVG serializer similarly supports native tooltips and data attributes on each region, so host pages can add their own interaction. The web app provides controls for glyph mode, spacing, and styling.

Research Impact Statement

Eunoia descends from eulerr, which has been distributed on CRAN since 2016 and has been cited in over 700 academic publications[2], predominantly in the life sciences. Since version 8.0, eulerr has used Eunoia in place of its C++ backend, so all current eulerr users also use Eunoia. Other packages depend on eulerr, and therefore on Eunoia, to draw area-proportional diagrams in their analyses. These include three Bioconductor genomics packages—cola, hicVennDiagram, and seqsetvis—and the CRAN package RulesTools. Six more packages—DOTSeq, highdir, IlluminaHumanMethylationEPICv2manifest, ISAnalytics, overviewR, and pcutils—use eulerr optionally and can therefore also invoke Eunoia.

The Python, Julia, and JavaScript packages and the web app extend the same underlying library to communities that previously had no access to ellipse-based area-proportional diagrams.

AI Usage Disclosure

JL wrote Eunoia's predecessor, eulerr, entirely without AI. For Eunoia, JL used generative AI tools including Claude Code Opus (versions 4.5, 4.8, and 5.0), Claude Code Fable 5, GitHub Copilot GPT 5.1 and 5.4, and Codex GPT 5.6-sol and GPT 6.1-sol. NML used Claude Code Opus 5.5. The authors used these tools to help write code, parts of the documentation, and unit tests and to review and edit the manuscript. NML performed user testing and detected bugs independently of AI. JL and NML reviewed, modified, and validated all AI-assisted content and made the final design and implementation decisions.

Acknowledgements

Eunoia builds on a decade of feedback from users of eulerr, whose bug reports and feature requests have shaped the fitting engine, and on Peter Gustafsson's contributions to the original eulerr package.

References

Chow, S. C. (2007). *Generating and drawing area-proportional Euler and Venn diagrams* [PhD thesis]. University of Victoria.

Eberly, D. (2016, November). *The area of intersecting ellipses*. Geometric Tools. https://www.geometrictools.com/Documentation/AreaIntersectingEllipses.pdf

Euler, L. (1802). *Letters of Euler to a German princess, on different subjects in physics and philosophy*. Murray; Highley.

[2] Google Scholar, accessed Aug 11, 2026

DRAFT

Frederickson, B. (2015, June 29). *A better algorithm for area proportional Venn and Euler diagrams*. https://www.benfrederickson.com/better-venn-diagrams/

Hansen, N., & Ostermeier, A. (1996). Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. *Proceedings of IEEE International Conference on Evolutionary Computation*, 312–317. https://doi.org/10.1109/ICEC.1996.542381

Junta, C. M., Sandrin-Garcia, P., Fachin-Saltoratto, A. L., Mello, S. S., Oliveira, R. D. R., Rassi, D. M., Giuliatti, S., Sakamoto-Hojo, E. T., Louzada-Junior, P., Donadi, E. A., & Passos, G. A. S. (2009). Differential gene expression of peripheral blood mononuclear cells from rheumatoid arthritis patients may discriminate immunogenetic, pathogenic and treatment features. *Immunology*, *127*(3), 365–372. https://doi.org/10.1111/j.1365-2567.2008.03005.x

Larsson, J. (2026). *Eunoia: Area-proportional Euler and Venn diagrams* (Version 1.9.0). https://doi.org/10.5281/zenodo.22042667

Larsson, J., & Gustafsson, P. (2018). A case study in fitting area-proportional Euler diagrams with ellipses using eulerr. *Proceedings of International Workshop on Set Visualization and Reasoning*, *2116*, 84–91. https://ceur-ws.org/Vol-2116/paper7.pdf

Micallef, L., Dragicevic, P., & Fekete, J.-D. (2012). Assessing the effect of visualizations on Bayesian reasoning through crowdsourcing. *IEEE Transactions on Visualization and Computer Graphics*, *18*(12), 2536–2545. https://doi.org/10.1109/TVCG.2012.199

Micallef, L., & Rodgers, P. (2014). eulerAPE: Drawing area-proportional 3-Venn diagrams using ellipses. *PLOS ONE*, *9*(7), e101717. https://doi.org/10.1371/journal.pone.0101717

Pérez-Silva, J. G., Araujo-Voces, M., & Quesada, V. (2018). NVenn: Generalized, quasi-proportional Venn and Euler diagrams. *Bioinformatics*, *34*(13), 2322–2324. https://doi.org/10.1093/bioinformatics/bty109

Richter-Gebert, J. (2011). *Perspectives on projective geometry: A guided tour through real and complex geometry*. Springer. https://doi.org/10.1007/978-3-642-17286-1

Tretyakov, K. (2024). *Matplotlib-venn: Area-weighted Venn diagrams for Python/matplotlib*. https://github.com/konstantint/matplotlib-venn

Wilkinson, L. (2012). Exact and approximate area-proportional circular Venn and Euler diagrams. *IEEE Transactions on Visualization and Computer Graphics*, *18*(2), 321–331. https://doi.org/10.1109/TVCG.2011.56

Wybrow, M., Rodgers, P., & Dib, F. K. (2021). Euler diagrams drawn with ellipses area-proportionally (Edeap). *BMC Bioinformatics*, *22*(1), 214. https://doi.org/10.1186/s12859-021-04121-8

DRAFT